

Discovery Executive Summary

Project: discovery-14-aug · Generated: 14/08/2026, 11:55:27

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 1 discovery analysis run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating
1	Security Analysis	—

1. Security Analysis

EXECUTIVE SUMMARY

The Klearcom platform has significant security weaknesses across both its PHP backend and Node.js dev-API. The most critical finding is the complete absence of authentication and authorization on all API routes — every endpoint (including data-mutating POST routes) is publicly accessible without any credential check. A wildcard CORS configuration (`allowed_origins: ['*']`) compounds this by allowing any origin to call the API. The backend uses PHP's unsafe `extract()` on unfiltered user input in `LegacyReportController`, enabling variable injection. The dev-API `bulk-import` endpoint accepts arbitrary payloads with zero validation or rate limiting. Hardcoded database credentials (`secret` / `root`) are committed in `docker-compose.yml`, `.env.example`, and the CI workflow. `APP_DEBUG=true` is set in committed configuration, which would leak stack traces and environment variables in production. No security headers (CSP, HSTS, X-Frame-Options) are configured on either the Nginx reverse proxy or the application. The frontend has no XSS sinks, no hardcoded secrets, and no browser-storage token issues, but lacks a Content Security Policy and uses a hardcoded `http://` fallback API URL. The CI pipeline has no SAST, dependency scanning, or branch protection evidence. Layers covered: backend (PHP/Laravel), dev-API (Node.js/Express), frontend (React/TypeScript), infrastructure (Docker/Nginx/CI).

6.1 Security Benchmark Ratings

#	Security KPI	Target	<span class="rating	<span class="rating	<span	Measured	Rating
			class="rating	rating-moderate">Moderate	class="rating		
					rating-high-		

			rating-good">Good		risk">High Risk		
H1	Critical Vulnerabilities	0	0	1	>1	3	High Risk
H2	High Vulnerabilities	0	<5	5–10	>10	5	Moderate
H3	Medium Vulnerabilities	low	<20	20–50	>50	3	Good
H4	Vulnerability Density	<0.5/KLOC	<0.5	0.5–1.0	>1.0	3.6/KLOC	High Risk
H5	OWASP Top 10 Compliance	>95%	>95%	80–95%	<80%	30% (7/10 categories with findings)	High Risk
H6	Critical/High Vulnerable Deps	0	0	1	>1	0	Good
H7	Outdated Dependencies	<10%	<10%	10–25%	>25%	<10%	Good
H8	End-of-Life Dependencies	0	0	1–5	>5	0	Good

6.5 Actions Required

Finding	Action	Rating	Priority
No Authentication/Authorization	Implement auth middleware (Sanctum/JWT) on all API routes; add login/registration; apply RBAC	High Risk	Critical
Wildcard CORS	Restrict <code>allowed_origins</code> to explicit frontend domain allow-list in both Laravel and Express	High Risk	Critical
Unsafe <code>extract()</code> on User Input	Replace all <code>extract()</code> calls with explicit array access in <code>LegacyReportController</code> and <code>LegacyDataMapper</code>	High Risk	Critical
Hardcoded Credentials	Use Docker secrets / env-var references; replace <code>.env.example</code> passwords with placeholders; use GitHub Actions secrets in CI	High Risk	High
APP_DEBUG Enabled	Set <code>APP_DEBUG=false</code> in <code>.env.example</code> and remove from <code>docker-compose.yml</code>	High Risk	High

No Security Headers	Add CSP, X-Frame-Options, X-Content-Type-Options, HSTS to Nginx config and frontend CSP meta tag	High Risk	High
No CSRF Protection	Implement token-based auth (inherent CSRF protection) or add VerifyCsrfToken middleware	High Risk	High
No Rate Limiting	Register RateLimiter in AppServiceProvider; apply throttle middleware; add express-rate-limit to dev-API	High Risk	High
Missing Frontend CSP	Add <code><meta http-equiv=\"Content-Security-Policy\"></code> to index.html; change fallback URL to HTTPS	Moderate	Medium
No SAST/Dependency Scanning in CI	Add <code>composer audit</code> , <code>npm audit</code> , PHPStan, and Dependabot to CI pipeline; enable branch protection	Moderate	Medium
No Security Audit Logging	Add structured logging for auth events, data mutations, and API access patterns	Moderate	Medium